

JAVA 17 OCP · YDS STUDY SERIES

Unit 04 · Technical Memory Notes · Core APIs

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu dosya eksiksiz [çift dilli Chapter 4 notunu](#), Core APIs için method ezberi yerine “object değişir mi, return type nedir, index sınırı nedir, zaman çizgisi nedir?” sorularıyla tamamlar. Dil çalışması: [vocabulary](#).

Kapsam denetimi

Ana çift dilli kaynak String pool, StringBuilder chaining, arrays, Math return types, Instant ve daylight saving konularını tam kapsar. Bu hafıza notu şu teknik ayrımları birlikte tekrar ettirir:

- String ve StringBuilder method'larının mutability farkı,
- array sort/search/compare/mismatch kurallarının birlikte görülmesi,
- Math method return type'lerinin sabitlenmesi,
- `LocalDate/LocalTime/LocalDateTime/ZonedDateTime/Instant` ayrımı,
- `Period` ile `Duration` ve DST etkisinin bağlanması.

1. String: sonuç döner, original değişmez

```
String name = " lion ";
name.trim();
System.out.println "[" + name + ""]; // [ lion ]
name = name.trim().toUpperCase();
System.out.println(name);           // LION
```

String immutable'dır. Bir method “değişmiş String” döndürüyorsa return value yeniden atanmadıkça original reference aynı object'i göstermeye devam eder.

Hafıza cümlesi: String'e emir verilmez; String'den yeni sonuç alınır.

Sınavlık String method haritası

Method	Ana kural
<code>length()</code>	UTF-16 code unit sayısı (<code>char</code> sayısı)
<code>charAt(i)</code>	valid index <code>0..length-1</code>
<code>indexOf(...)</code>	bulunamazsa <code>-1</code>
<code>substring(begin,end)</code>	begin inclusive, end exclusive
<code>replace(...)</code>	tüm eşleşmeleri yeni String'de değiştirir
<code>strip()</code>	Unicode-aware whitespace temizler
<code>trim()</code>	daha eski, <code><= U+0020</code> yaklaşımı
<code>indent(n)</code>	line indentation'ı değiştirir, line ending normalize eder
<code>translateEscapes()</code>	String içindeki escape text'ini yorumlar

```
String s = "abcdef";
System.out.println(s.substring(2, 5)); // cde
```

Geçerli sınırlar içinde `begin == end` empty String verir. Negative index, end'in length'i aşması veya begin'in end'den büyük olması runtime exception üretir.

2. String pool, identity ve equality

```
String a = "cat";
String b = "ca" + "t";           // compile-time constant, pooled
String c = new String("cat");

System.out.println(a == b);      // true
System.out.println(a == c);      // false
System.out.println(a.equals(c)); // true
```

- `==` reference identity,
- `String.equals()` character content,
- `intern()` pooled canonical reference döndürür.

Constant expression olmayan String concatenation yeni bir String nesnesi üretir. Compile-time constant String expression ise intern edilir ve pool'daki reference'ı kullanır.

3. StringBuilder: aynı object üzerinde çalışır

```
var sb = new StringBuilder("cat");
sb.append("s").insert(0, "big ").reverse();
```

Önemli method'lar: `append`, `insert`, `delete`, `deleteCharAt`, `replace`, `reverse`, `setCharAt`, `substring`.

- Çoğu mutating method aynı builder reference'ını döndürerek chaining sağlar.
- `substring()` bir `String` döndürür ve builder'ı değiştirmez.
- `StringBuilder.equals()` content comparison override etmez; identity davranışı gösterir.
- `capacity()` storage alanıdır; `length()` kullanılan character sayısıdır.

Hafıza cümlesi: String yeni sayfa açar; StringBuilder aynı tahtayı silip yazar.

4. Arrays: type sabit, length sabit

```
int[] numbers = new int[3]; // [0, 0, 0]
String[] names = new String[2]; // [null, null]
```

- Array object'tir; `length` field'dır, method değildir.
- Element type ve length creation'dan sonra değişmez.
- Multidimensional arrays aslında array-of-arrays yapısıdır; rows farklı length'lere sahip olabilir.

```
int[][] grid = new int[2][];
grid[0] = new int[3];
grid[1] = new int[1];
```

Arrays algoritmaları

Method	Hatırlanacak sonuç
<code>Arrays.sort(a)</code>	original array mutata edilir
<code>Arrays.binarySearch(a, key)</code>	array önce compatible order ile sorted olmalı
<code>Arrays.compare(a, b)</code>	lexicographic karşılaştırma
<code>Arrays.mismatch(a, b)</code>	ilk farklı index; tamamen aynıysa -1

Binary search bulunamayan key için `-(insertion point) - 1` döndürür. Unsorted array üzerinde sonuç güvenilir değildir.

5. Math API return-type kartı

Method	Return type / aralık
<code>Math.min, Math.max</code>	overload'a göre numeric type
<code>Math.round(float)</code>	<code>int</code>
<code>Math.round(double)</code>	<code>long</code>
<code>Math.ceil, Math.floor</code>	<code>double</code>
<code>Math.pow</code>	<code>double</code>
<code>Math.random</code>	<code>double, 0.0 <= x < 1.0</code>

`ceil` number line'da pozitif sonsuza, `floor` negatif sonsuza gider. Bu nedenle negative values için “decimal kısmı silme” ile aynı değildir.

6. Date-time type seçimi

Type	Ne taşır?	Zone/offset var mı?
<code>LocalDate</code>	tarih	Hayır
<code>LocalTime</code>	saat	Hayır
<code>LocalDateTime</code>	tarih + saat	Hayır
<code>ZonedDateTime</code>	tarih + saat + zone	Evet
<code>Instant</code>	UTC timeline point	UTC tabanlı

Bu class'ların constructor'ları yerine `now()`, `of()`, `parse()` gibi factory methods kullanılır. Hepsi immutable'dir:

```
java.time.LocalDate date = java.time.LocalDate.of(2024, 1, 31);
date.plusDays(1);
System.out.println(date); // 2024-01-31
date = date.plusDays(1); // 2024-02-01
```

Invalid date creation compile olmaz diye düşünülmemelidir; çoğu durumda kod derlenir ve `DateTimeException` runtime'da oluşur.

7. Period, Duration ve DST

- `Period`: year/month/day tabanlı date amount.
- `Duration`: seconds/nanos tabanlı time amount.

```
java.time.Period oneDay = java.time.Period.ofDays(1);
java.time.Duration twentyFourHours = java.time.Duration.ofHours(24);
```

`Period` date-based types ile, `Duration` time/instant-based types ile doğal çalışır. `Unsupported unit` eklemek runtime'da `UnsupportedTemporalTypeException` üretebilir.

Zoned timeline'da DST geçişinde “bir takvim günü” ile “tam 24 saat” farklı local time sonuçları verebilir:

```
zdt.plus(Period.ofDays(1))      → aynı local saat hedeflenir
zdt.plus(Duration.ofHours(24)) → timeline'a tam 24 saat eklenir
```

Hafıza cümlesi: Period takvime, Duration saate bakar.

`DateTimeFormatter` text ↔ temporal dönüşümünde kullanılır. Parsed text hedef type'ın gerekli alanlarını taşımalıdır; yalnız date, `LocalDateTime` için yetmez.

OCP için 20 saniyelik analiz sırası

- 1 Object immutable mı mutable mı; method original'ı değiştiriyor mu?
- 2 Method'un return type'ı nedir ve sonuç yeniden atanmış mı?
- 3 Index sınırı ve array search için sorting ön koşulu doğru mu?
- 4 Date-time type/amount gerekli date, time, zone ve unit bilgisini taşıyor mu?

Aktif hatırlama · Özgün çalışma soruları

- 1 `String.trim()` dönüş değeri değişkene atanırsa özgün String nesnesi değişir mi?
- 2 `Arrays.binarySearch()` için temel ön koşul nedir?
- 3 `Math.round(2.4)` return type'ı nedir?
- 4 DST gününde `Period.ofDays(1)` ile `Duration.ofHours(24)` neden ayrışabilir?

Cevaplar

- 1 Hayır; String immutable'dır.
- 2 Array aynı comparison order ile sorted olmalıdır.
- 3 Literal `double` olduğu için `long`.
- 4 Biri local calendar date'i, diğeri timeline'da tam saat miktarını iletir.